Date

Expt. No. 5. Page No. 15

Aim:- To write a C program to check whether a string is a keyword or not.

Theory:- Keywords are reserved words in C with predefined meaning.
The strcmp() function is used for comparison. The function compares two strings character by character.

Algorithm

1. Start
2. input string
3. Compare with keyword list
4. If match → keyword.
5. Else → Not keyword.
6. Stop.

Program

```
#include<stdio.h>
#include<string.h>
 int main()
{
 char str[20];
 char keywords[7][10] =
   {"int", "float", "if", "else", "while", "for",
    "return"};
int i, flag=0;
 printf("Enter a string: ");
```

Teacher's Signature

```
scanf("%s", str);
for(i=0; i<7; i++){
    if(strcmp(str, keywords[i])==0)
{
 flag=1;
 break;
}
}
;f(flag)
    printf("Keywords\n");
return 0;
}
```

Viva Question:-

Q1 What defines a keyword in a programming language?
Ans A keyword is a reserved word that has a predefined meaning in a programming language. Examples in C are int, if, else, while etc.

Q2 How do you implement keyword checking efficiently?
Ans Store the keyword in an array & compare the input string with each keyword using strcmp(). If any comparison returns 0, the input is identified as a keyword.

Q3 Why can't keyword be used as identifiers?
Ans Keyword already have a special meaning to the compiler.

Teacher's Signature

Therefore, they cannot be used as variable, function or other identifier names.

Q4 How does your program handle case sensitivity?

Ans The program uses strcmp(), which is case-sensitive.
So, int is recognised as a keyword, but Int & INT are not.

Q5 What is the difference between a keyword & a reserved word?

Ans A keyword is a word with a predefined meaning & is used in the syntax of a language. A reserved word is kept aside by the language & generally cannot be used as a identifier.

Teacher's Signature 31/8

Date

Expt. No. 6. Page No. 18.

Aim:- To write a C program to count the total number of keywords present in a file.

Algorithm

1. Start
2. Open file in read mode
3. Initialize keyword counter
4. Read words from file
5. Compare each word with keyword list
6. If match found - increment counter
7. Repeat until EOF
8. Print total keywords
9. Close file.
10. Stop

Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// List of C keywords
const char *keywords[] =
{"int", "float", "char", "if", "else", "while", "for",
"return", "break", "continue", "void", "double"};

#define KEYWORD_COUNT (sizeof(keywords)/
sizeof(keywords[0]))
```

Teacher's Signature

```
// function to check if a word is a keyword
int is_keyword(const char *word)
{
    for(int i = 0; i < KEYWORD_COUNT; i++)
    {
        if(strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int main()
{
    FILE *fp;
    char word[100];
    int count = 0;
    fp = fopen("input.txt", "r");
    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }
// Read words one by one
while(fscanf(fp, "%99s", word) != EOF)
{
// Remove punctuation if present
    for(int i = 0; word[i]; i++)
{
    if(ispunct(word[i]))
{   word[i] = '\0';
    break;
}
}
}
```

```
        if (iskeyword(word))
        {
            count++;
            printf("Keyword found : %s\n", word);
        }
    }
    printf("\n Total keywords = %d\n", count);
    fclose(fp);
    return 0;
}
```

Viva Questions

Q1 How does your program identify keywords within a source file?

Ans The program reads the source file character by character & identifies words using delimeters like spaces & symbols. It then compares each words with a predefined list of keywords & counts the matches.

Q2 How do you avoid counting keywords inside string literals?

Ans The program tracks whether it is inside single or double quotes while reading the source file. Any words found inside string literals are ignored, preventing them from being counted as keywords.

Teacher's Signature

Date

Expt. No. Page No. 21.

Qs 2 How do you handle comments in your keyword counting?

As The program detects single-line (//) & multi-line (/*...*/) comments & ignores their contents. This ensures the keywords written inside comments are not counted.

Qs 4 What is the difference between counting keywords & counting reserved words?

As Keywords are predefined words that have special meaning in a programming language, such as if, while, & return. Reserved words that cannot be used as identifiers; in many languages, keywords are a subset of reserved words.

Qs 5 How does your program handle pre-processor directives?

Ans The program identifies pre-processor directives, such as #include & #define, by checking for the '#' symbol at the beginning of a line. It handles them seperately to avoid incorrectly counting keywords inside these directives.

Teacher's Signature

Date

Expt. No. 7. Page No. 22.

Aim Write a program to count the number of operators in a file.

Algorithm

1. Start
2. Open the source file in read mode.
3. Check whether the current character can be an operator
4. Read the next character to identify two character operators such as ++, ==, >= && etc.
5. Increment the counter whenever an operator is found.
6. Continue until EOF.
7. Close the file & display the total number of operators.

Program

```
#include <stdio.h>
int main() {
    FILE *fp;
    char ch, next;
    int count = 0;
    fp = fopen("input.c", "r");
    if (fp == NULL) {
    printf("file cannot be opened.\n");
    return 1;
}
```

Teacher's Signature

```
while ((ch = fgetc(fp)) != EOF) {
    if ( ch == '+' || ch == '-' || ch == '=' ||
     ch == '!' || ch == '<' || ch == '>' ||
     ch == '&' || ch == '|' || ch == '*' ||
     ch == '/' || ch == '%' )
    {
     next = fgetc(fp);
    if ( ( ch == '+' && next == '+' ) ||
        ( ch == '-' && next == '-' ) ||
        (ch == '=' && next == '=' ) ||
        (ch == '!' && next == '=' ) ||
        (ch == '<' && next == '=' || next == '<' )) ||
        (ch == '>' && next == '=' || next == '>' )) ||
        (ch == '&' && next == '&' ) ||
        (ch == '|' && next == '|' ) ||
        (ch == '+' && next == '=' ) ||
        (ch == '-' && next == '=' ) ||
        (ch == '*' && next == '=' ) ||
        (ch == '/' && next == '=' ) ||
        (ch == '%' && next == '=' ) )
    {
     count ++;
    }
else
    if( ch == '+' || ch == '-' || ch == '=' ||
        ch == '!' || ch == '<' || ch == '>' ||
        ch == '*' || ch == '/' || ch == '%' )
    {
     count ++;
    }
```

Teacher's Signature _____

```
        if(next1 != EOF)
          ungetc(next, fp);
        }
      }
    }
    fclose(fp);
    printf("Total number of operators = %d\n",
        count);
    return 0;
}
```

Viva Questions

Q1 What is the purpose of this program?

Ans The purpose of this program is to read a source file character by character & identify & count the operators present in that file.

Q2 How does the ~~purpose~~ program identify & signed character operator?

Ans The program compares the current character with known operator symbols such as +, -, *, /, %, =, <, > & increments the counter when a match is found.

Teacher's Signature

Q3 What is the time complexity of this program.
Ans If the file contains n characters, the program generally takes O(n) time because it scans the file once.

Q4 How does the program identify a two character operator?
Ans If first reads the current character & then reads the next character. If the combination forms an operator such as ==, ++, >= or && the counter is increased by one.

Q5 What happens if the file cannot be opened?
Ans The program checks whether fp == NULL. If true, it displays "file cannot be opened" & terminates the program.

Teacher's Signature